



PREDICTIVE MAINTENANCE MODEL DOCUMENTATION

(Jackson Steel Plant System)



1. SYSTEM OVERVIEW

This predictive maintenance module is designed to:

- Analyze real-time machine sensor + operational data
 - Predict:
 - **Failure within 30 days (Binary Classification)**
 - **Remaining Useful Life (Regression)**
 - Trigger:
 - Maintenance alerts
 - Email notifications via n8n
 - Dashboard updates
-



2. MODELS IMPLEMENTED

◆ Model 1: Failure Prediction Model

Attribute	Value
Type	Binary Classification
Algorithm	XGBoost Classifier
Output	<code>failure_within_30_days</code> (0 or 1)
Accuracy	~99.9%
Purpose	Detect imminent machine failure

◆ Model 2: RUL Prediction Model

Attribute	Value
Type	Regression
Algorithm	XGBoost Regressor
Output	RUL (Remaining Useful Life in days)
R ² Score	~0.73
MAE	~23
Purpose	Estimate machine lifespan



3. INPUT FEATURES (CRITICAL)

The model expects **exactly these features** as input:



Core Features

Feature	Description
temperature	Machine temperature
vibration	Mechanical vibration
pressure	Internal pressure
rpm	Rotations per minute
power_consumption	Power usage
load	Machine load
usage_hours	Total operational hours
wear_level	Mechanical wear (0–100)
corrosion_level	Corrosion level
efficiency	Machine efficiency (%)
output_rate	Production output
error_count	Number of system errors

warning_count	Number of warnings
oil_level	Oil level
coolant_level	Coolant level

Engineered Features

Feature	Formula
stress_index	$\text{vibration} \times \text{load}$
energy_efficiency	$\frac{\text{output_rate}}{\text{power_consumption}}$
health_score	$100 - \text{degradation factors}$

Categorical Features (One-Hot Encoded)

Feature	Values
machine_type	Furnace, Motor, Pump
plant_location	Plant_A, Plant_B, Plant_C

👉 These must be **one-hot encoded before inference**

4. INPUT FORMAT (VERY IMPORTANT)

Expected Input: JSON Example

```
{  
  "temperature": 85,  
  "vibration": 72,  
  "pressure": 50,  
  "rpm": 1500,  
  "power_consumption": 220,  
  "load": 0.8,  
  "usage_hours": 3200,  
  "wear_level": 65,  
}
```

```
"corrosion_level": 40,  
"efficiency": 75,  
"output_rate": 90,  
"error_count": 12,  
"warning_count": 6,  
"oil_level": 45,  
"coolant_level": 50,  
"stress_index": 57.6,  
"energy_efficiency": 0.4,  
"health_score": 60,  
"machine_type_Furnace": 1,  
"machine_type_Motor": 0,  
"machine_type_Pump": 0,  
"plant_location_Plant_A": 1,  
"plant_location_Plant_B": 0,  
"plant_location_Plant_C": 0  
}
```



5. OUTPUT FORMAT

◆ Failure Model Output

```
{  
  "failure_within_30_days": 1  
}
```

◆ RUL Model Output

```
{  
  "RUL": 42  
}
```



Combined Output (Recommended)

```
{  
  "failure_within_30_days": 1,  
  "RUL": 42,  
  "maintenance_required": 1  
}
```



6. ALERT LOGIC (FOR n8n)

Trigger Conditions:

IF failure_within_30_days == 1:
 send_alert()

IF RUL < 30:
 schedule_maintenance()



Email Alert Example

- Subject:  Machine Failure Risk Detected
 - Body:
 - Machine ID
 - Risk Level
 - RUL
 - Key contributing factors
-



7. EXPLAINABILITY (SHAP)

The system uses SHAP to explain predictions.

Example Explanation:

“Failure predicted due to high vibration, increased wear level, and rising error count.”



8. PREPROCESSING REQUIREMENTS

Before inference:

MUST DO:

1. Drop:

- `machine_id`
 - 2. Encode:
 - `machine_type`
 - `plant_location`
 - 3. Ensure:
 - Same feature order as training
 - 4. No missing values
-

COMMON MISTAKES (IMPORTANT)

-  Not encoding categorical features
-  Missing engineered features
-  Wrong column order
-  Feeding raw data directly

 Any of these will break predictions

9. INFERENCE PIPELINE

Flow:

Input Data
↓
Preprocessing (encoding + features)
↓
Model Prediction
↓
Output (Failure + RUL)
↓
Trigger (n8n / Dashboard)

10. DEPLOYMENT USAGE

Backend (FastAPI / Node):

- Load `.pkl` models
- Accept JSON input

- Run prediction
 - Return output
-

n8n Workflow:

New Row →

Trigger →

Send to Model API →

Get Prediction →

IF failure → Send Email



11. FINAL SYSTEM CAPABILITIES

- ✓ Real-time monitoring
 - ✓ Failure prediction
 - ✓ Maintenance optimization
 - ✓ Explainable AI
 - ✓ Automated alerting
-



FINAL NOTE (FOR PRESENTATION)

Say this:

“Our system doesn’t just predict failures — it understands machine health, explains risks, and enables proactive maintenance through real-time intelligence.”